AutoAudit — Manual de ejecución, estudio y explicación del código

Proyecto: Sistema de auditoría automatizada
Asignatura: Bases de Datos I
Base del alcance: enunciado «AutoAudit», Universidad Mariano Gálvez de Guatemala, septiembre de 2026
Tecnologías implementadas: PostgreSQL 18, PL/pgSQL, PHP 8.3, PDO, Apache, Bootstrap 5.3.8 y Docker Compose.
Propósito del manual: instalar el proyecto, observar el efecto de cada trigger, reconocer las variables y comprender qué hace cada bloque.

---

1. ¿Qué vas a construir y qué problema resuelve?

En un sistema normal, cuando editas el teléfono de un cliente, el teléfono anterior desaparece de la fila del cliente. No sabes necesariamente quién lo cambió, cuándo ocurrió ni qué valor tenía antes.

AutoAudit es un segundo conjunto de tablas y funciones que almacena una evidencia de ese cambio. Permite decidir **qué tabla**, **qué columna** y **qué operaciones** vale la pena vigilar. No tienes que escribir un trigger diferente manualmente para cada columna: una función crea los disparadores cuando guardas las reglas.

Ejemplo:

Acción	Tabla `public.clientes`	Resultado en AutoAudit
INSERT	Se crea Ana	Guarda la fila nueva y los campos configurados
UPDATE	Teléfono: `5555-1001` → `5555-9999`	Guarda valor anterior y nuevo
UPDATE	Nombre actualizado con el mismo contenido	Si no hay cambio real en columnas auditadas, no registra evento
DELETE	Se elimina Ana	Guarda `OLD` completo aunque ya no exista en clientes

La **base de datos contiene la lógica de auditoría**; PHP se encarga de las pantallas, el inicio de sesión y el contexto (usuario, estación, IP). Bootstrap aporta presentación visual, pero no sustituye las funciones SQL.

1.1 ¿Qué es un trigger?

Un *trigger* o disparador es una instrucción que PostgreSQL ejecuta **automáticamente** cuando ocurre un evento en una tabla. En este proyecto se usan:

- `AFTER INSERT`: se ejecuta después de crear una fila.
- `AFTER UPDATE`: se ejecuta después de actualizar una fila.
- `AFTER DELETE`: se ejecuta después de eliminar una fila.

El trigger recibe los datos especiales `NEW` (versión nueva) y `OLD` (versión anterior), más `TG_OP` (nombre de operación) y `TG_TABLE_NAME` (tabla que lo ejecutó).

Diferencia importante: el trigger no es la bitácora. El trigger **detecta y llama** a `registrar_cambio()`; esa función **escribe los datos** en las tablas de bitácora.

1.2 PL/SQL frente a PL/pgSQL

El enunciado llama al lenguaje de los triggers «PL/SQL». **PL/SQL** es la denominación habitual del lenguaje procedural de Oracle. En **PostgreSQL 18** el lenguaje utilizado aquí se llama **PL/pgSQL** (`LANGUAGE plpgsql`). Se conserva el objetivo académico del enunciado, usando el lenguaje propio del motor requerido.

---

2. Estructura de carpetas, en palabras sencillas

Elemento	Qué es	¿Cuándo lo abres?
`.env.ejemplo`	Modelo de credenciales, puertos y primer admin	Antes de ejecutar
`compose.yaml`	Orquesta un servidor PostgreSQL y un servidor web	Para iniciar el sistema
`app/Dockerfile`	Instala PHP, Apache y `pdo_pgsql`	Para entender el servidor
`app/public/index.php`	Rutas, formularios y lógica principal de pantallas	Para estudiar frontend/backend
`app/src/BaseDatos.php`	Conexión a PostgreSQL con PDO	Para estudiar la conexión
`app/src/Seguridad.php`	Inicio de sesión, roles, CSRF y contexto del trigger	Para estudiar autenticación

Elemento	Qué es	¿Cuándo lo abres?
`app/src/Consultas.php`	Diccionario de datos y consultas de historial	Para estudiar SELECT con parámetros
`app/src/Presentacion.php`	Encabezado, menú y pie de página Bootstrap	Para estudiar la interfaz
`01_roles.sh`	Crea el usuario de BD para la app web	Solo instalación inicial
`02_estructura.sql`	Crea todas las tablas e índices	Para estudiar DDL
`03_funciones.sql`	Crea funciones, comparación OLD/NEW y triggers dinámicos	**Parte más importante**
`04_permisos.sql`	Limita lo que la aplicación puede modificar	Para estudiar roles y permisos
`05_admin.sh`	Crea el primer usuario administrador de la web	Solo instalación inicial
`base_de_datos/pruebas/`	Pruebas funcionales, carga y catálogo	Para validar el sistema
`diagramas/`	UML y ER en `.puml`, `.dot`, `.svg` y `.png`	Para documentación / explicación

Los archivos se enumeran en el orden en que resulta más sencillo comprenderlos.

---

3. Instalación desde cero en Ubuntu 26.04 LTS

3.1 Requisitos previos

Necesitas una computadora con:

- Ubuntu 26.04 LTS, o Windows/macOS con una instalación compatible de Docker.
- Docker Engine (o Docker Desktop) y complemento **Docker Compose** que funcione con `docker compose`.
- Espacio de disco suficiente para descargar las imágenes de PostgreSQL y PHP.
- Conexión a Internet **durante la primera instalación** y para cargar el CSS/JS de Bootstrap desde CDN.
- Una terminal: en Ubuntu puedes usar `Ctrl + Alt + T`.

Para instalar Docker en Ubuntu se recomienda seguir la **guía oficial del fabricante** para la versión actual de Ubuntu: `https://docs.docker.com/engine/install/ubuntu/`. El repositorio o paquete

disponible cambia con la versión del sistema; por eso se comprueba primero en lugar de suponer que una orden específica de paquetes funcionará.

Verifica ambas herramientas:

```
docker --version
docker compose version
```

Si alguna orden no existe, primero instala o configura Docker. **No hace falta instalar PHP, Apache ni PostgreSQL manualmente** cuando se utiliza este método.

3.2 Descomprimir y entrar al directorio

Guarda el ZIP en Descargas. Desde la terminal:

```
cd ~/Descargas
unzip AutoAudit_Proyecto_Completo.zip
cd AutoAudit_Proyecto
ls -la
```

Si descomprimiste el ZIP mediante el explorador de archivos, abre una terminal dentro de `AutoAudit_Proyecto`. `ls -la` debe mostrar `compose.yaml`, `.env.ejemplo`, `app/` y `base_de_datos/`.

3.3 Preparar variables de entorno

```
cp .env.ejemplo .env
nano .env
```

Modifica por lo menos las claves:

```
POSTGRES_PASSWORD=TU_CLAVE_PRIVADA_DE_POSTGRES
AUTOAUDIT_CLAVE_APP=OTRA_CLAVE_PRIVADA_PARA_PHP
AUTOAUDIT_ADMIN_USUARIO=administrador
AUTOAUDIT_ADMIN_CLAVE=TU_CLAVE_DE_ACCESO_WEB
```

El usuario de PostgreSQL `postgres` administra el motor; `autoaudit_aplicacion` es un usuario distinto, creado con la clave `AUTOAUDIT_CLAVE_APP`. El usuario `administrador` inicia sesión **en el navegador**, no directamente en PostgreSQL.

Nunca subas el archivo `.env` a un repositorio público. Ya está incluido en `.gitignore`.

3.4 Construir e iniciar el sistema

Ejecuta desde la carpeta raíz:

```
docker compose up -d --build
```

La primera vez Docker descarga las imágenes, compila la extensión de PHP y crea automáticamente las tablas, funciones, roles y usuario inicial.

Para saber qué contenedores arrancaron:

```
docker compose ps
```

Debes ver los servicios `bd` y `web`. Si aparecen errores:

```
docker compose logs --tail=100 bd
docker compose logs --tail=100 web
```

Abre **`http://localhost:8080`**. Si cambiaste `PUERTO_WEB` en `.env`, usa ese puerto.

Escribe las credenciales de **`AUTOAUDIT_ADMIN_USUARIO`** y **`AUTOAUDIT_ADMIN_CLAVE`**. El administrador podrá entrar a las cuatro pantallas: Panel, Configuración, Laboratorio e Historial.

3.5 Acceso opcional desde pgAdmin

Puedes conectar pgAdmin al servidor de PostgreSQL usando:

Parámetro	Valor
Host	`localhost`
Puerto	`5433` (o `PUERTO_POSTGRES`)
Base de datos	`autoaudit`
Usuario	`postgres`
Contraseña	El valor de `POSTGRES_PASSWORD`

El esquema `auditoria` contiene el sistema de auditoría; el esquema `public` contiene las tablas ficticias de `clientes` y `productos`.

Si pgAdmin está instalado **dentro de otro contenedor**, `localhost` ya no apunta al servicio de PostgreSQL; usa el nombre de red `bd` y el puerto interno `5432`.

3.6 ¿Cómo detener y reiniciar?

```
# Detiene los contenedores, conservando los datos.
docker compose down

# Vuelve a iniciarlos conservando los datos.
docker compose up -d
```

Nunca ejecutes `docker compose down -v` si quieres conservar el proyecto: el parámetro `-v` destruye el volumen de PostgreSQL. Los scripts de inicialización `.sql` y `.sh` se ejecutan en una base **nueva**, no cada vez que reinicias Docker.

---

4. Primer ejercicio guiado: crear y comprobar un trigger

Paso 1: seleccionar la tabla

En el menú **Configuración**, elige `public.clientes` y presiona **Ver columnas**. Las columnas no se escribieron manualmente dentro de la pantalla; el código consulta el catálogo de PostgreSQL.

Paso 2: seleccionar reglas

Marca las casillas siguientes:

Columna	INSERT	UPDATE	DELETE
nombre	Sí	Sí	Sí
telefono	Sí	Sí	Sí
correo	No	Sí	No

Deja las otras casillas desmarcadas. Haz clic en **Guardar reglas y generar triggers**.

Paso 3: comprobar los triggers desde SQL

Conecta por pgAdmin o por terminal:

```
docker compose exec bd psql -U postgres -d autoaudit
```

Dentro de `psql` ejecuta:

```
SELECT trigger_name, event_object_table, event_manipulation
FROM information_schema.triggers
WHERE trigger_name LIKE 'tr_autoaudit_%'
ORDER BY event_object_table, trigger_name;
```

Deberás encontrar tres triggers asociados a la tabla `clientes`: uno por INSERT, otro por UPDATE y otro por DELETE.

Paso 4: generar una modificación

Abre el menú **Laboratorio**. Crea un cliente ficticio, por ejemplo:

```
Nombre: Carla Pérez
Correo: carla.ejemplo@example.com
Teléfono: 5000-1111
```

Después haz clic en **Editar** y cambia el teléfono a `5000-2222`. Finalmente, puedes eliminar ese cliente de prueba.

Paso 5: comprobar OLD y NEW

En **Historial**, filtra por tabla `clientes`, operación `UPDATE` y columna `telefono`. Presiona **Detalle**. Debes ver el número anterior y el nuevo, el usuario, fecha, IP y las instantáneas completas de la fila.

Este es el objetivo principal de AutoAudit: no solo actualizar datos, sino conservar evidencia de **qué pasó**.

---

5. Bloques de código SQL, explicados uno por uno

5.1 `02_estructura.sql` — Crear las entidades

Bloque A: esquema de auditoría.

```
CREATE SCHEMA IF NOT EXISTS auditoria;
CREATE EXTENSION IF NOT EXISTS pgcrypto;
```

- `CREATE SCHEMA` separa las tablas de auditoría de los datos de negocio de `public`.
- `pgcrypto` permite crear el hash bcrypt del usuario inicial. No guarda la clave en texto plano.

Bloque B: usuarios y estaciones. `auditoria.usuarios` incluye el nombre, el hash de contraseña, el rol y si está activo. `auditoria.estaciones_trabajo` guarda una huella construida desde IP y navegador y la fecha de último acceso. Esa huella **no garantiza** identificar un dispositivo físico: dos sesiones con la misma IP y user-agent pueden compartirla.

Bloque C: configuración de la tabla. `tablas_auditadas` contiene `nombre_esquema`, `nombre_tabla`, `activa` y `version_configuracion`. `columnas_auditadas` mantiene las columnas y `reglas_auditoria` mantiene las operaciones por columna.

Bloque D: bitácora. `bitacora_eventos` registra el evento completo, y `bitacora_detalles` guarda **una fila por cada columna vigilada que participó en ese evento**. Esta separación ayuda a consultar por columna sin tener que buscar dentro de documentos JSON grandes.

Bloque E: índices. Los índices sobre fecha, usuario y tabla hacen más eficientes las búsquedas recurrentes, aunque aumentan trabajo y almacenamiento al insertar auditorías.

Bloque F: laboratorio. `public.clientes` y `public.productos` son tablas completamente ficticias para practicar; no contienen clientes reales.

5.2 `03_funciones.sql` — Función A: `obtener_o_crear_estacion()`

```
INSERT INTO auditoria.estaciones_trabajo
    (identificador, nombre, direccion_ip, agente_navegador)
VALUES (...)
ON CONFLICT (identificador) DO UPDATE
    SET ultimo_acceso = CURRENT_TIMESTAMP
RETURNING id_estacion INTO identificador_estacion;
```

- **Entrada:** huella SHA-256, nombre de estación, IP y cadena del navegador.
- `ON CONFLICT` evita crear estaciones duplicadas cuando existe la misma huella.
- `RETURNING` devuelve `id_estacion`, usado por el trigger como referencia.
- `SECURITY DEFINER` significa que esta función usa privilegios de su propietario; por eso debe validar datos y mantener un `search_path` controlado.

5.3 Función B: `registrar_cambio()` — El corazón del proyecto

B1. Validación del trigger.

```
SELECT id_tabla, version_configuracion, activa
INTO configuracion_tabla
FROM auditoria.tablas_auditadas
WHERE id_tabla = identificador_tabla
  AND nombre_esquema = TG_TABLE_SCHEMA
  AND nombre_tabla = TG_TABLE_NAME;
```

`TG_ARGV[0]` contiene el ID de configuración. `TG_TABLE_SCHEMA` y `TG_TABLE_NAME` identifican la tabla real. Se valida que el evento pertenezca a la tabla esperada y que la auditoría esté activa.

B2. Crear representaciones de OLD y NEW.

```
IF TG_OP = 'INSERT' THEN
    registro_viejo := NULL;
    registro_nuevo := to_jsonb(NEW);
ELSIF TG_OP = 'DELETE' THEN
    registro_viejo := to_jsonb(OLD);
    registro_nuevo := NULL;
ELSE
    registro_viejo := to_jsonb(OLD);
    registro_nuevo := to_jsonb(NEW);
END IF;
```

- `to_jsonb` transforma una fila, cuyas columnas pueden variar según la tabla, en un objeto JSONB.
- INSERT no tiene valor anterior; DELETE no tiene valor nuevo.

- Esta elección hace posible utilizar una **sola función** para distintas tablas de negocio.

B3. Decidir si existe un cambio que valga la pena guardar.

```
AND (TG_OP <> 'UPDATE' OR
    (registro_viejo -> columna.nombre_columna)
    IS DISTINCT FROM
    (registro_nuevo -> columna.nombre_columna))
```

- `->` extrae el valor JSONB de una columna.
- `IS DISTINCT FROM` compara correctamente también cuando hay NULL.
- Si el usuario hace UPDATE pero las columnas auditadas no cambian, la función no inserta evento.

B4. Encontrar la clave primaria. El código consulta `pg_index` y `pg_attribute` para descubrir qué columnas forman la clave primaria y construye un JSONB. Se admiten claves primarias compuestas. Si una tabla no tiene PK, se guarda `{}`; en ese caso debes interpretar el evento usando las instantáneas de fila.

B5. Identificar al autor. El código consulta la información que PHP envía mediante `set_config('autoaudit.usuario_id', ..., true)` y las claves de estación e IP. Si el cambio proviene de SQL ejecutado directamente en la BD, esas variables pueden estar vacías; en ese caso se conserva `session_user` de PostgreSQL.

B6. Insertar encabezado. `INSERT INTO auditoria.bitacora_eventos ... RETURNING id_evento` guarda toda la información del cambio e identifica el registro recién creado.

B7. Insertar detalles. El bucle `FOR columna_regla IN ... LOOP` recorre las columnas que tienen regla activa y crea sus entradas OLD/NEW en `bitacora_detalles`.

¿Y si la función falla? Al ejecutarse dentro de la misma transacción SQL que INSERT/UPDATE/DELETE, un error de trigger revierte también el cambio origen. Esto evita cambios de negocio confirmados sin su bitácora, pero exige estudiar bien permisos y errores.

5.4 Función C: `generar_triggers()` — Generación automática

La función recibe un `identificador_tabla` y consulta cuál es el esquema y nombre exacto de la tabla. Verifica que sea una tabla ordinaria de `public` y que las columnas todavía existan.

```
FOREACH tipo_operacion IN ARRAY
        ARRAY['INSERT','UPDATE','DELETE']
LOOP
    -- Para cada tipo se reemplaza el trigger, si debe existir.
    EXECUTE format('DROP TRIGGER IF EXISTS %I ON %I.%I', ...);
    -- Si hay reglas activas se crea otro trigger AFTER.
    EXECUTE format('CREATE TRIGGER %I AFTER %s ON %I.
        %I ...', ...);
END LOOP;
```

- `FOREACH` recorre los tres eventos de la lista cerrada.
- `DROP TRIGGER IF EXISTS` evita duplicados al cambiar la configuración.
- `format('%I', ...)` **protege los identificadores SQL** (nombres de tablas y triggers) y los cita correctamente.
- `format('%L', ...)` cita el ID transmitido como argumento literal.
- `%s` se usa **solo** con uno de tres valores constantes controlados (`INSERT`, `UPDATE`, `DELETE`), nunca directamente con texto del usuario.
- Se crea un trigger por tabla y operación activa, pero todos usan la función genérica `registrar_cambio()`.

Una limitación explícita: el proyecto no soporta por diseño tablas particionadas, auditoría DDL, SELECT o `TRUNCATE`. Estos casos requieren un diseño adicional.

5.5 Función D: `configurar_tabla()` — Convertir selección en reglas

Recibe un esquema, tabla y un JSONB como este:

```
{
  "nombre": ["INSERT", "UPDATE", "DELETE"],
  "telefono": ["UPDATE", "DELETE"],
  "correo": ["UPDATE"]
}
```

D1. Valida que la tabla exista en el catálogo del DBMS. **D2.** Inserta o actualiza `tablas_auditadas`, sube la versión y bloquea la fila durante el proceso. **D3.** Reemplaza las reglas vigentes después de comprobar que cada columna y cada operación existen. **D4.** Ejecuta `generar_triggers()`.

Todo ocurre en una operación SQL: si los nuevos triggers no pueden instalarse, la modificación de reglas tampoco debe quedar confirmada.

5.6 Función E: `cambiar_estado()`

```
UPDATE auditoria.tablas_auditadas
   SET activa = estado_deseado,
       version_configuracion = version_configuracion + 1
 WHERE id_tabla = identificador_tabla;
PERFORM auditoria.generar_triggers(identificador_tabla);
```

Pausar retira los triggers mediante el generador. **Reactivar** los reconstruye siguiendo las reglas almacenadas. Ninguna de estas acciones borra `bitacora_eventos` ni `bitacora_detalles`.

---

6. Bloques PHP: qué hace el servidor y por qué

6.1 `BaseDatos.php` — Conectar una sola vez

```php
$conexion_compartida = new PDO(

        "pgsql:host={$servidor};port={$puerto};dbname={$nombre_base_datos}",
    $nombre_usuario,
    $contrasena,
    [PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION]
);
```

- `PDO`: interfaz común de PHP para conectarse a bases de datos.
- `pgsql:`: el controlador es PostgreSQL.
- `$servidor`, `$puerto`, `$nombre_base_datos`: ubicación de la BD.
- `$nombre_usuario` / `$contrasena`: credenciales del rol limitado `autoaudit_aplicacion`.
- `ATTR_ERRMODE => ERRMODE_EXCEPTION`: si PostgreSQL rechaza una consulta, PHP puede capturar el error.

No confundas el rol de la base con el usuario que inicia sesión en AutoAudit: son identidades distintas.

6.2 `Seguridad.php` — Protección de formularios

- `usuario_autenticado()`: devuelve el usuario web guardado en la sesión.
- `requerir_sesion()`: bloquea visitas no autenticadas.
- `requerir_administrador()`: restringe la configuración de triggers.
- `escapar_html()`: transforma texto potencialmente peligroso para mostrarlo sin ejecutar etiquetas HTML.
- `token_csrf()` y `comprobar_csrf()`: impiden aceptar formularios que no tengan el token de sesión esperado.
- `avisar()` e `ir_a()`: muestran mensajes y redirigen después del POST.

6.3 `Seguridad.php` — Asociar usuario web con un trigger

```
$conexion->beginTransaction();
// crear/obtener id_estacion
// set_config('autoaudit.usuario_id', '...', true)
// realizar INSERT/UPDATE/DELETE sobre clientes
$conexion->commit();
```

¿Por qué se usa una transacción? La variable `autoaudit.usuario_id` se define con el último argumento `true`, que la limita **a esa misma transacción**. Cuando el UPDATE dispara el trigger, PostgreSQL ya puede saber quién actuó en la interfaz.

Si ocurre una excepción, `rollBack()` deshace los cambios. `hash('sha256', ...)` crea una huella para estación usando IP y User-Agent. La IP se lee desde `REMOTE_ADDR`, sin confiar ciegamente en cabeceras `X-Forwarded-For`.

6.4 `Consultas.php` — Descubrir tablas reales

```
SELECT espacio.nspname AS esquema, clase.relname AS tabla
FROM pg_catalog.pg_class clase
JOIN pg_catalog.pg_namespace espacio
  ON espacio.oid = clase.relnamespace
WHERE espacio.nspname='public' AND clase.relkind='r';
```

No se usa una lista fija de tablas. PostgreSQL guarda metadatos sobre sus tablas en `pg_catalog`. `relkind='r'` significa tabla ordinaria.

El método `obtener_columnas_tabla()` utiliza `pg_attribute` y `format_type` para recuperar nombre y tipo de cada columna.

6.5 `Consultas.php` — Búsquedas SQL parametrizadas

```
$condiciones[] = 'evento.id_usuario = :usuario';
$parametros['usuario'] = $filtros['usuario'];
$consulta = $conexion->prepare($consulta_sql);
$consulta->bindValue(':usuario', $valor, PDO::PARAM_INT);
```

`:usuario` es un **parámetro**, no una cadena concatenada al SQL. PDO transporta el valor separadamente, para reducir el riesgo de inyección SQL. La consulta también maneja `tabla`, `operacion`, `columna`, `desde`, `hasta` y paginación.

6.6 `Presentacion.php` — Bootstrap

Este archivo dibuja el menú, la estructura HTML, los mensajes y el pie de página. Importa Bootstrap CSS/JS 5.3.8 desde CDN y carga `estilos.css` local para el aspecto de AutoAudit.

Bootstrap ofrece componentes visuales: `container`, `row`, `col-md-4`, `card` o panel, `btn`, `table`, `modal`, `alert`, etc. Por ejemplo:

```
<button class="btn btn-primary">Guardar reglas y generar
        triggers</button>
```

El atributo `class` controla el estilo. **No ejecuta SQL**: la acción ocurre cuando el formulario llega al servidor PHP.

6.7 `index.php` — El controlador que conecta todo

Este es el flujo de un formulario:

1. El navegador envía `POST`, `accion` y `token_csrf`.
2. `index.php` valida el token y la sesión.
3. Si `accion = guardar_configuracion`, verifica el rol de administrador.
4. Comprueba la tabla y las columnas contra el catálogo real.
5. Convierte la selección a JSONB.
6. Ejecuta `SELECT auditoria.configurar_tabla(...)` usando parámetros.
7. PostgreSQL guarda reglas y genera triggers automáticamente.
8. PHP redirige a la configuración y muestra un mensaje.

El laboratorio hace algo similar con `crear_cliente`, `actualizar_cliente` y `eliminar_cliente`, pero rodea el cambio de una transacción con contexto de usuario.

Importante sobre el CSV: `exportar` se procesa **antes** de imprimir HTML para enviar cabeceras de descarga correctas. Se exporta como máximo 10,000 eventos por descarga; es una protección de consumo de memoria, no una garantía de exportación masiva sin límites.

---

7. Diccionario de variables: nombres en español

Nombre	Dónde aparece	Significado
`identificador_tabla`	PL/pgSQL	ID de la configuración de tabla
`configuracion_tabla`	PL/pgSQL	Fila actual de tabla y su versión
`registro_viejo`	PL/pgSQL	OLD como JSONB
`registro_nuevo`	PL/pgSQL	NEW como JSONB
`registro_clave`	PL/pgSQL	Clave primaria de la fila modificada
`numero_columnas`	PL/pgSQL	Cantidad de columnas que ameritan un evento
`identificador_evento`	PL/pgSQL	ID del encabezado de bitácora recién creado

Nombre	Dónde aparece	Significado
`columna_regla`	PL/pgSQL	Columna actual del bucle de detalles
`tipo_operacion`	PL/pgSQL	INSERT, UPDATE o DELETE
`nombre_disparador`	PL/pgSQL	Nombre del trigger generado
`reglas_recibidas`	PL/pgSQL	Objeto JSONB de reglas por columna
`eventos_columna`	PL/pgSQL	Lista de operaciones de una columna
`conexion_compartida`	PHP	Objeto PDO reutilizado en la solicitud
`usuario_autenticado`	PHP / sesión	Usuario web que inició sesión
`accion_solicitada`	PHP	Valor que distingue qué formulario se envió
`reglas_validas`	PHP	Reglas comprobadas antes de transmitirlas a PostgreSQL
`filtros`	PHP	Tabla, usuario, operación, columna y fechas buscadas
`condiciones`	PHP	Fragmentos WHERE fijos y seleccionados por la app
`parametros`	PHP	Valores transmitidos separadamente a SQL
`direccion_ip`	PHP y SQL	IP asociada al cambio
`identificador_estacion`	PHP	SHA-256 del contexto de navegador/IP
`token_csrf`	PHP	Token que protege los formularios POST

Los nombres especiales `NEW`, `OLD`, `TG_OP`, `TG_RELID`, `TG_ARGV`, `CURRENT_TIMESTAMP` y las palabras clave SQL permanecen en inglés porque **son nombres reservados del lenguaje**, no variables que convenga traducir.

---

8. Comprender los diagramas

8.1 Casos de uso (01)

El administrador puede configurar tablas, generar triggers y usar el laboratorio. Un usuario de rol `consulta` entra al historial y exporta reportes. Las funciones de configuración se protegen por permisos web. El laboratorio, en esta demostración, está accesible a usuarios autenticados.

8.2 Clases (02)

Representa las entidades como clases con atributos y relaciones. Un objeto de `TablaAuditada` tiene varias `ColumnaAuditada`; cada una tiene varias `ReglaAuditoria`. Un evento contiene varios detalles.

8.3 Secuencia de configuración (03)

Explica qué sucede **en orden temporal** cuando se marcan casillas: administrador → navegador → PHP → función SQL → generador de triggers → PostgreSQL.

8.4 Secuencia de auditoría (04)

Explica la actualización del cliente desde el navegador y cómo, dentro de la misma transacción, el trigger compara OLD/NEW y escribe bitácora.

8.5 Actividad (05)

Muestra decisiones: ¿administrador? ¿existe regla activa? ¿la columna realmente cambió? Su propósito es entender el flujo, no el SQL exacto.

8.6 Componentes (06)

Separa el servidor web PHP del motor PostgreSQL y representa sus conexiones y funciones principales, desplegados en contenedores sobre el anfitrión Ubuntu.

8.7 Entidad-relación (07)

Se centra en PK, FK y cardinalidades de las siete tablas del esquema auditoría. **Es distinto del diagrama de clases UML**: describe el almacenamiento real en el motor. Para tipos y restricciones completos, abre `Diccionario_de_Datos.md`.

Todos los archivos `.puml` se pueden editar en PlantUML. Los `.dot` generan imágenes con Graphviz. Los `.svg` son ampliables sin perder calidad; las `.png` se insertan fácilmente en documentos.

---

9. Pruebas de validación (importantes para tu exposición)

9.1 Ejecutar las pruebas desde la terminal

Desde el directorio del proyecto:

```
docker compose exec -T bd psql -U postgres -d autoaudit -v
        ON_ERROR_STOP=1 < base_de_datos/pruebas/
        01_pruebas_funcionales.sql
```

Las pruebas verifican:

1. Que se generan **tres triggers** para la tabla de prueba.
2. Que `INSERT` registra un evento.
3. Que un `UPDATE` sin modificación real no genera evento nuevo.
4. Que un `UPDATE` diferente conserva los valores OLD y NEW.
5. Que `DELETE` preserva la instantánea aunque la fila desaparezca.
6. Que pausar la auditoría elimina los triggers vigentes sin borrar el historial.

Al finalizar, se ejecuta **ROLLBACK**, para que estas pruebas no alteren permanentemente tus reglas y datos.

9.2 Consultas para revisar manualmente

```
-- Qué reglas configuraste.
SELECT tabla.nombre_tabla, columna.nombre_columna,
        regla.operacion
FROM auditoria.tablas_auditadas tabla
JOIN auditoria.columnas_auditadas columna ON
        columna.id_tabla=tabla.id_tabla
JOIN auditoria.reglas_auditoria regla ON
        regla.id_columna=columna.id_columna
ORDER BY tabla.nombre_tabla, columna.nombre_columna,
        regla.operacion;

-- Últimos 10 eventos (encabezados).
SELECT id_evento, nombre_tabla, operacion, fecha_evento,
        clave_primaria
FROM auditoria.bitacora_eventos
ORDER BY id_evento DESC LIMIT 10;

-- Historial detallado de la columna telefono.
SELECT evento.fecha_evento, evento.operacion,
       detalle.valor_anterior, detalle.valor_nuevo
FROM auditoria.bitacora_eventos evento
JOIN auditoria.bitacora_detalles detalle ON
        detalle.id_evento=evento.id_evento
WHERE detalle.nombre_columna='telefono'
ORDER BY evento.fecha_evento DESC;
```

9.3 Medir rendimiento

```
docker compose exec -T bd psql -U postgres -d autoaudit -v
        ON_ERROR_STOP=1 < base_de_datos/pruebas/
        02_prueba_rendimiento.sql
```

El comando `EXPLAIN (ANALYZE, BUFFERS, SUMMARY)` muestra el tiempo que PostgreSQL invirtió procesando inserciones por lotes. El script compara dos volúmenes de registros (100 y 1,000) dentro de una transacción descartada con ROLLBACK.

No se puede afirmar que el impacto sea «insignificante» sin medirlo. Debes registrar tus tiempos reales, la versión del sistema y las características del equipo. Si quieres cuantificar sobrecarga, repite una prueba comparable con triggers desactivados y otra con activación.

---

10. Ejercicios para que comprendas el código y puedas modificarlo solo

Ejercicio 1 — Identificación. Busca en `registrar_cambio()` las líneas donde se convierten `OLD` y `NEW` a JSONB. Anota qué valor es NULL para INSERT y DELETE.

Ejercicio 2 — Reglas. En `configurar_tabla()` observa cómo se valida el nombre de columna consultando `pg_attribute`. Intenta enviar una columna inexistente y verifica que PostgreSQL rechaza la configuración.

Ejercicio 3 — Cambios reales. Desde el laboratorio, edita un cliente sin cambiar el teléfono. Luego edita únicamente el teléfono. Comprueba cuántos eventos UPDATE aparecen si solo auditaste teléfono.

Ejercicio 4 — Nueva tabla. Abre PostgreSQL y crea una tabla sencilla `public.proveedores (id_proveedor BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY, nombre TEXT NOT NULL)`. La tabla aparecerá en Configuración porque se consulta el catálogo. Puedes crear reglas y auditar operaciones hechas en SQL. Para que una página PHP la edite, necesitarás permisos y un formulario nuevo.

Ejercicio 5 — Cambio de nombre. Cambia el nombre de una columna origen en una tabla de laboratorio; intenta regenerar triggers. Explica por qué el sistema advierte que una regla se refiere a una columna inexistente.

Ejercicio 6 — Filtro SQL. Ubica `construir_consulta_historial()` y explica por qué utiliza `:usuario`, `:columna` y `:limite` en vez de concatenar entradas de texto del navegador.

Ejercicio 7 — Seguridad. Intenta conectarte como `autoaudit_aplicacion` y modificar directamente `auditoria.bitacora_eventos`. Debe rechazarse porque no se concedieron permisos directos de escritura sobre la bitácora.

Ejercicio 8 — Versionado. Configura la tabla, cambia la selección de operaciones y revisa `version_configuracion` en nuevos eventos. Explica cómo permite saber qué reglas estaban vigentes cuando se registró un cambio.

---

11. Problemas frecuentes y soluciones

Síntoma	Causa posible	Cómo revisar
`docker: command not found`	Docker no instalado o no configurado	Revisar `docker --version` y la guía oficial
`docker compose` no existe	Falta el complemento Compose	Instalar y revisar `docker compose version`
Puerto `8080` ocupado	Otro servicio utiliza ese puerto	Cambiar `PUERTO_WEB` en `.env`
Puerto `5433` ocupado	Ya existe otro PostgreSQL o servicio	Cambiar `PUERTO_POSTGRES`
Error 500 en PHP	Error en credenciales, inicialización o SQL	`docker compose logs --tail=100 web bd`
Usuario web no ingresa	Clave equivocada o volumen inicializado con otra clave	Revisar valores usados en **primer** arranque
No aparece un evento UPDATE	No seleccionaste esa columna o no cambió su valor	Revisar Configuración y Detalle
Hay reglas, pero no triggers	Tabla pausada, columna eliminada o error al instalar	Ejecutar consulta `information_schema.triggers`
Bootstrap se ve sin formato	El navegador no accede a CDN	Verificar Internet o colocar copia local autorizada
Error `permission denied` al probar otra tabla	El rol app no tiene permiso DML sobre esa tabla	Usar psql como admin o conceder solo permisos necesarios
Estación/usuario nulos en un cambio SQL	Operación hecha directamente desde PostgreSQL	El contexto web solo se conoce si se transmite
`relation ... does not exist`	No se inicializó bien la BD o tabla mal nombrada	Revisar logs iniciales de `bd` y `\dt auditoria.*`
No se actualiza un SQL corregido	El volumen ya tenía la BD creada	Aplicar una migración manual; reiniciar contenedores no repite scripts

Precaución: si vas a modificar funciones SQL después de la primera inicialización, entra a la BD y ejecuta el archivo de funciones para reemplazarlas; NO borres datos con `down -v` sin saber sus consecuencias:

```
docker compose exec -T bd psql -U postgres -d autoaudit -v
        ON_ERROR_STOP=1 < base_de_datos/inicializacion/
        03_funciones.sql
```

---

12. Conceptos de seguridad y límites de la demostración

- Las consultas de la interfaz utilizan **PDO parametrizado**; las partes dinámicas de triggers usan `format('%I', ...)` para nombres de objetos verificados contra el catálogo.
- Las contraseñas web se guardan como hash **bcrypt**, no texto plano.
- Las acciones POST usan token **CSRF** y las respuestas HTML escapan texto de usuarios.
- La cuenta `autoaudit_aplicacion` **no puede modificar directamente** las tablas de bitácora, pero puede invocar funciones de configuración si obtiene acceso SQL a esa cuenta. Por eso también se debe proteger el servidor y las credenciales: en el diseño académico la verificación de rol para configurar está en PHP.
- La interfaz no admite `TRUNCATE`, `SELECT`, creación/eliminación de tablas ni cambios de esquema como eventos de auditoría.
- Las instantáneas JSONB almacenan **todos los valores de fila**, incluso columnas no seleccionadas, para preservar registros borrados. En un sistema real, esto requiere políticas de privacidad, retención y acceso a datos sensibles.
- No existe una forma infalible de identificar un usuario humano solo desde PostgreSQL si varios empleados comparten la misma cuenta de conexión y su aplicación no comunica identidad.
- Se recomienda en producción HTTPS, credenciales rotadas, gestor de secretos, redes privadas, roles de BD por privilegio, copias de seguridad verificadas y auditoría protegida fuera del mismo servidor.
- Los contenedores Docker están publicados en **localhost**, apropiado para laboratorio. No publiques los puertos a Internet sin revisar autenticación, TLS y controles de red.
- El reporte CSV incluye datos que pueden ser sensibles. El código neutraliza celdas que parecen fórmulas de hoja de cálculo, pero se deben aplicar controles de acceso adecuados.
- Las pruebas incluidas usan `ASSERT` y `ROLLBACK` para minimizar el impacto sobre datos y configuraciones locales; no se deben aplicar indiscriminadamente a una BD de producción.

---

13. ¿Cómo publicar los fuentes en GitHub o GitLab?

El proyecto exige entregar el **enlace** a un repositorio. El ZIP es el código listo para subir, pero publicar el repositorio requiere iniciar sesión en **tu propia cuenta**.

Desde la raíz del proyecto, luego de crear un repositorio vacío en GitHub o GitLab:

```
git init
git add .
git commit -m "AutoAudit: base de datos, interfaz y
        documentacion"
git branch -M main
git remote add origin URL_DE_TU_REPOSITORIO
git push -u origin main
```

Verifica que `.env` **NO** aparezca en `git status` ni en el repositorio. `.env.ejemplo` sí se debe compartir, porque no contiene claves reales (cambia sus contraseñas antes de usarlo).

En la entrega añade el enlace del repositorio y el PDF del manual. También puedes agregar capturas del Panel, Configuración, Laboratorio, Historial, Detalle y pruebas con la salida de la terminal.

---

14. Guion recomendado para defender el proyecto

1. **Necesidad:** «Cuando alguien modifica o elimina un dato, necesitamos un historial verificable».
2. **Arquitectura:** «PHP + Bootstrap son la interfaz; PostgreSQL 18 ejecuta la lógica principal».
3. **Configuración:** «No escribimos manualmente cada trigger; elegimos tablas, columnas y operaciones».
4. **Automatización:** «`configurar_tabla()` valida los metadatos y `generar_triggers()` crea los triggers AFTER».
5. **Auditoría:** «`registrar_cambio()` compara OLD y NEW y guarda encabezado más detalles».
6. **Seguridad:** «El usuario web puede consultar, mientras que las tablas de bitácora tienen permisos restringidos».
7. **Pruebas:** «Demostramos INSERT, UPDATE, DELETE, valores anteriores y posteriores, filtros y CSV».
8. **Limitación conocida:** «Es una implementación académica para tablas ordinarias `public`, no una plataforma universal de auditoría de producción».

Preguntas típicas que podría hacer el docente

¿Qué ocurre si edito una columna no auditada? Si no cambia ninguna columna con regla UPDATE, no se crea evento.

¿Para qué se usa JSONB? Para almacenar instantáneas de filas con estructuras distintas; por ejemplo, clientes tiene campos diferentes a productos.

¿Por qué se usa un trigger AFTER? Porque el requisito pide capturar cambios tras la operación; el trigger ve el valor resultante y guarda la evidencia dentro de la misma transacción.

¿En qué se diferencia el generador de hacer manualmente un CREATE TRIGGER? El generador consulta una tabla de configuración y construye automáticamente el SQL para las operaciones activas de cada tabla.

¿Qué pasa con un DELETE? La fila desaparece de la tabla origen, pero su `registro_anterior` sigue en `bitacora_eventos`.

¿Qué pasa si desactivo auditoría? Se eliminan los triggers vigentes para esa tabla; los eventos históricos permanecen.

¿Qué pasa si falla la inserción en la bitácora? PostgreSQL cancela la sentencia de negocio dentro de la transacción para conservar consistencia; la aplicación muestra error y realiza ROLLBACK.

---

15. Checklist de entrega

- [ ] El ZIP fue descomprimido y `docker compose up -d --build` funciona.
- [ ] Puedes iniciar sesión con tu administrador.
- [ ] Configuraste `public.clientes` y verificaste los triggers en SQL.
- [ ] Probaste INSERT, UPDATE y DELETE desde el laboratorio.
- [ ] Observaste `registro_anterior`, `registro_nuevo`, usuario, IP y fecha.
- [ ] Probaste filtro por usuario/columna y descargaste CSV.
- [ ] Ejecutaste `01_pruebas_funcionales.sql` y obtuviste el resultado esperado.
- [ ] Ejecutaste la medición de rendimiento y guardaste los tiempos observados.
- [ ] Incluiste los diagramas UML y el diagrama entidad-relación en tu documentación.
- [ ] Revisaste `Diccionario_de_Datos.md` y `Matriz_de_Cumplimiento.md`.
- [ ] Publicaste el código en tu GitHub/GitLab y copiaste su enlace.
- [ ]

Confirmaste qué pide tu docente sobre capturas, fecha final y formato de exposición.

Referencias técnicas para ampliar

- Documentación PostgreSQL 18, funciones PL/pgSQL y SQL dinámico: `https://www.postgresql.org/docs/18/plpgsql-statements.html`
- Documentación PostgreSQL 18, CREATE TRIGGER: `https://www.postgresql.org/docs/18/sql-createtrigger.html`
- Documentación PostgreSQL 18, visión general de triggers: `https://www.postgresql.org/docs/18/triggers.html`
- Bootstrap 5.3, guía oficial: `https://getbootstrap.com/docs/5.3/getting-started/introduction/`
- Docker, instalación oficial para Ubuntu: `https://docs.docker.com/engine/install/ubuntu/`

Fin del manual. Recomendación de estudio: ejecuta el capítulo 4, después lee las funciones del capítulo 5 con el archivo SQL abierto en paralelo. No memorices la sintaxis antes de comprender cómo viaja un cambio desde el navegador a OLD/NEW y después a la bitácora.